

Solidity

1st lesson

1. 1st key concepts

- **Solidity**: a programming language for writing smart contracts on Ethereum.
- **Smart contract**: a program that runs automatically on the blockchain.
- **Simple example**: store and read a message ("Hello World").

2. Basic Contract Structure

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MyContract {
5     // variables
6     // functions
7 }
```

- **SPDX-License-Identifier** : specifies the license (MIT here).
- **pragma solidity** : defines the compiler version.
- **contract [MyContract]** : defines the contract.

3. Basic Data Types

Type	Description
string	text / character string
uint	positive integer
bool	true / false
address	Ethereum address
bytes	binary data

4. Functions & Visibility

- **public** : accessible from outside the contract
- **private** : accessible only inside the contract
- **view** : reads data, does not modify the blockchain
- **pure** : neither reads nor modifies the blockchain

5. Key commands

- **require(condition, "message")** : checks a condition, reverts transaction if false
- **return** : returns a value from a function

6. Tools for beginners

- **Remix IDE** : easy for testing and deploying code
- **Advanced frameworks**: Hardhat, Foundry, Truffle

7. Hello World example's explanation

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3 contract HelloWorld
4 {
5     string private stateVariable = "Hello World";
6     function GetHelloWorld() public view returns ( string memory)
7     {
8         return stateVariable;
9     }
10 }
11 }
```

stateVariable

- Type: string
- Stores the message "Hello World"
- Visibility: private (only accessible via a function)

GetHelloWorld()

- public view : accessible from outside, read-only
- Returns the value of stateVariable

8. Deployment & Testing

- Open Remix IDE
- Create a file HelloWorld.sol
- Copy and paste the code
- Compile with Solidity ^0.8.0 (or 0.7+)
- Deploy the contract, call GetHelloWorld(), returns "Hello World"

9. Tips

- Declare compiler version (**pragma**) and license (**SPDX-License-Identifier**).
- **private + public view** = standard way to control access to data.